



ADS01: AVL Trees、Splay Trees 与 Amortized Analysis

依据 Materials/Student Slides/ADS01AVL_Stu.ppt 按 PPT 顺序整理。页码为课件页码。

1. 速查术语索引 (Glossary)

English	中文	一句话解释
Binary Search Tree (BST)	二叉搜索树	左子树键值小于节点，右子树键值大于节点。
Height	树高	根到最深叶节点的最长路径长度，课件空树取 -1 。
Height-balanced tree	高度平衡树	左右子树高度差绝对值不超过 1，且子树递归满足该条件。
AVL tree	AVL 树	满足高度平衡条件的二叉搜索树。
Balance Factor (BF)	平衡因子	$h_L - h_R$ ，AVL 节点取值只能是 $-1, 0, 1$ 。
Single rotation	单旋转	通过一次局部旋转修复 LL 或 RR 型失衡。
Double rotation	双旋转	通过两次局部旋转修复 LR 或 RL 型失衡。
LL rotation	LL 旋转	左-左型失衡对应的右旋修复。
RR rotation	RR 旋转	右-右型失衡对应的左旋修复。
LR rotation	LR 旋转	左-右型失衡，先左旋后右旋。
RL rotation	RL 旋转	右-左型失衡，先右旋后左旋。
Splay tree	伸展树	访问节点后通过旋转将其移动到根的自调整 BST。
Splaying	伸展	将被访问节点沿路径旋转到根的过程。
Zig	Zig 旋转	父节点为根时，对节点与父节点做一次旋转。

English	中文	一句话解释
Zig-zag	Zig-zag 旋转	节点与父、祖父方向相反时进行的双旋转。
Zig-zig	Zig-zig 旋转	节点与父、祖父方向相同时进行的旋转调整。
Amortized analysis	摊还分析	对连续操作序列的总成本进行平均界定。
Aggregate analysis	聚合分析	先界定整个操作序列的总成本，再除以操作数。
Accounting method	记账法	用摊还成本差额保存 credit，支付未来成本。
Potential method	势能法	用状态势能变化修正每次操作的实际成本。
Potential function	势能函数	为数据结构状态赋值、表示已积累支付能力的函数。
Credit	Credit / 余额	摊还成本超过实际成本时保存的支付额度。
Subtree size	子树大小	以节点为根的子树节点数，包含该节点本身。
Rank	秩	课件中为 $R(i) = \log S(i)$ ，用于 Splay 势能分析。
Amortized cost	摊还成本	$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$ 。

2. 按 PPT 顺序梳理：核心知识与推理

2.1 从普通 BST 到 AVL 树：为什么要控制树高（PPT 1-3）

课件先从二叉搜索树出发。BST 的搜索沿根到目标节点的路径进行，因此：

$$T_p = O(h)$$

其中 T_p 是一次搜索的时间, h 是树高。普通 BST 的树高可能退化到 $O(N)$, 使搜索、插入和删除都退化为线性时间。

课件用月份序列展示两种 BST: 同样的关键字, 树越偏斜, 平均搜索路径越长。示例中较平衡的树平均搜索时间为 3.5 或 3.1, 偏斜树为 6.5。因此需要在更新树的同时维护结构平衡。

2.2 AVL 的定义与平衡因子 (PPT 4)

课件将 AVL 树定义为高度平衡的 BST。空树高度平衡, 并约定:

$$h(\emptyset) = -1$$

对非空节点, 若左右子树高度分别为 h_L, h_R , 则需要满足:

$$|h_L - h_R| \leq 1$$

平衡因子定义为:

$$BF(v) = h_L - h_R$$

因此 AVL 树中每个节点都必须满足:

$$BF(v) \in \{-1, 0, 1\}$$

插入或删除后, 沿更新路径检查 BF; 一旦出现 -2 或 2 , 就根据失衡方向进行局部旋转。

2.3 AVL 插入与旋转: 从实例到一般形式 (PPT 5-8)

RR 与 LL: 单旋转

课件先用 Mar, May, Nov 演示 RR 情况: Nov 落在失衡节点 Mar 的右子树的右子树中, 因此执行 RR 修复, 即对失衡节点做一次左旋。

RR: A 的右子树的右子树插入 -> 对 A 左旋

LL: A 的左子树的左子树插入 -> 对 A 右旋

旋转只改变局部父子关系, 必须保持所有子树的中序顺序, 因此不会破坏 BST 性质。课件还提醒: A 不一定是整棵树的根, 旋转也可能发生在内部子树。

LR 与 RL：双旋转

LR: A 的左子树的右子树插入

先对 A 的左孩子左旋, 再对 A 右旋

RL: A 的右子树的左子树插入

先对 A 的右孩子右旋, 再对 A 左旋

课件中的一般图用 $A_L, A_R, B_L, B_R, C_L, C_R$ 表示局部子树。旋转后, 这些子树的相对中序顺序保持不变, 但节点 A, B, C 的层级被重新安排。

插入完成后需要沿插入路径回到根, 更新高度并重新计算 BF。课件后续示例说明, 即使不需要继续重构树, 多个祖先节点的 BF 也可能发生变化, 因此实现时可以为每个节点维护 height 字段。

2.4 AVL 的高度为什么是对数级 (PPT 9-10)

设 n_h 为高度为 h 的高度平衡树的最少节点数。为使节点数尽可能少, 根的两个子树高度必须为 $h-1$ 与 $h-2$, 因此:

$$n_h = n_{h-1} + n_{h-2} + 1$$

这与 Fibonacci 递推相同:

$$F_0 = 0, \quad F_1 = 1, \quad F_i = F_{i-1} + F_{i-2} \quad (i > 1)$$

由初值可得:

$$n_h = F_{h+2} - 1, \quad h \geq 0$$

又因为:

$$F_i \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^i$$

所以:

$$n_h \approx \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{h+2} - 1$$

对上式取对数得到：

$$h = O(\ln n)$$

因此 AVL 的树高是对数级，搜索路径为 $O(\log n)$ ，这是 AVL 相比普通 BST 的核心优势。

2.5 Splay 树的目标：允许局部变差，控制连续操作（PPT 11-16）

Splay 树不要求每个时刻都严格高度平衡，而是保证任意 M 个连续操作的总时间为：

$$O(M \log N)$$

其中 N 是节点数。单次操作仍可能花费 $O(N)$ ，但一长段操作的摊还时间为 $O(\log N)$ 。

基本思想是：节点被访问后，通过一系列旋转把它推到根。这样，最近访问过的节点更容易再次访问，同时访问路径上的许多节点深度会大致减半。

课件先展示只把访问节点旋到根的做法可能导致：

$$T(N) = O(N^2)$$

因此 Splay 必须使用成组旋转，而不是简单地逐层向上旋转。

2.6 Splay 的三类旋转（PPT 14-16）

设被访问节点为 X ，父节点为 P ，祖父节点为 G 。

1. **Zig**：若 P 是根，旋转 X 与 P 一次。
2. **Zig-zag**：若 X 与 P 、 P 与 G 的方向相反，连续进行两次旋转。
3. **Zig-zig**：若两个方向相同，围绕 G 进行结构调整，使 X 一次上升两层。

X 为根	-> 结束
P 为根	-> Zig
P 不是根且方向相反	-> Zig-zag
P 不是根且方向相同	-> Zig-zig

不断重复判断，直到 X 到达根。课件强调，伸展不仅把 X 移到根，也大致压缩访问路径上其余节点的深度。

2.7 Splay 删除 (PPT 17)

1. 查找 X ，并通过 Splay 使 X 成为根。
2. 删除根 X ，得到左右子树 T_L, T_R 。
3. 在 T_L 中执行 FindMax，将左子树最大元素移到 T_L 的根；该节点没有右孩子。
4. 将 T_R 作为该根的右孩子。

选择 T_L 的最大元素，是因为它大于 T_L 中所有其他元素且小于 T_R 中所有元素，接上 T_R 后仍保持 BST 有序性。

2.8 摊还分析的三种方法 (PPT 18)

课件把目标表述为：任意 M 个连续操作的总成本至多为 $O(M \log N)$ ，这称为摊还时间界。

聚合分析	直接证明整段序列的总成本
记账法	给高成本操作预存 credit，支付未来成本
势能法	用数据结构状态的势能变化表达 credit

课件强调：

$$\text{worst-case bound} \geq \text{amortized bound} \geq \text{average-case bound}$$

摊还分析不依赖概率，因此不能把摊还界直接解释成随机输入下的平均时间。

2.9 聚合分析：Stack 的 MultiPop (PPT 19)

MultiPop(k, S) 的实际成本是弹出的元素数量：

$$T = \min(\text{sizeof}(S), k)$$

从空栈开始，在长度为 n 的 Push、Pop、MultiPop 序列中，每个元素最多被压入一次、弹出一次。因此总成本为 $O(n)$ ，摊还成本为：

$$T_{\text{amortized}} = \frac{O(n)}{n} = O(1)$$

2.10 记账法：把成本差额保存为 credit (PPT 20-21)

令 c_i 为实际成本， $\hat{c}_i$ 为摊还成本。必须保证：

$$\sum_{i=1}^n \hat{c}_i \geq \sum_{i=1}^n c_i$$

课件给出：

$$c_i(\text{Push}) = 1, \quad c_i(\text{Pop}) = 1, \quad c_i(\text{MultiPop}) = \min(\text{sizeof}(S), k)$$

并设定：

$$\hat{c}_i(\text{Push}) = 2, \quad \hat{c}_i(\text{Pop}) = 0, \quad \hat{c}_i(\text{MultiPop}) = 0$$

每次 Push 支付实际成本 1 后留下 1 个 credit；之后 Pop 或 MultiPop 消耗这些 credit。这样 MultiPop 虽然实际可能弹出多个元素，但不需要临时支付新的摊还成本，总摊还成本仍为 $O(n)$ 。

2.11 势能法：把 credit 写成状态变化 (PPT 22-23)

设 D_i 为第 i 次操作后的数据结构状态，势能函数为 $\Phi(D_i)$ 。定义：

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

对 n 次操作求和：

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$

课件要求势能函数通常在初始状态取得最小值，且：

$$\Phi(D_i) \geq 0 = \Phi(D_0)$$

对栈取：

$$\Phi(D_i) = \text{sizeof}(D_i)$$

于是：

- Push 使势能增加 1，所以 $\hat{c}_i = 1 + 1 = 2$ 。
- Pop 使势能减少 1，所以 $\hat{c}_i = 1 - 1 = 0$ 。
- 若 MultiPop 实际弹出 k' 个元素，势能减少 k' ，所以 $\hat{c}_i = k' - k' = 0$ 。

因此总摊还成本为 $O(n)$ ，每次摊还成本为 $O(1)$ 。势能法把记账法中的 credit 统一写成状态

函数。

2.12 Splay 的势能函数与摊还证明 (PPT 24-25)

课件对 Splay 树定义：

$$\Phi(T) = \sum_{i \in T} \log S(i)$$

其中 $S(i)$ 是包含节点 i 的子树大小，并定义节点秩：

$$R(i) = \log S(i)$$

因此：

$$\Phi(T) = \sum_{i \in T} R(i)$$

选择子树大小而不是直接使用树高，是因为旋转时局部子树大小会发生可计算的变化，秩差可以抵消旋转的实际成本。

课件记 R_1 、 R_2 分别为旋转前后的秩，给出：

- Zig:

$$\hat{c}_i = 1 + R_2(X) - R_1(X) + R_2(P) - R_1(P)$$

$$\hat{c}_i \leq 1 + R_2(X) - R_1(X)$$

- Zig-zag:

$$\hat{c}_i \leq 1 + 3(R_2(X) - R_1(X))$$

- Zig-zig:

$$\hat{c}_i = 2 + R_2(X) - R_1(X) + R_2(P) - R_1(P) + R_2(G) - R_1(G)$$

$$\hat{c}_i \leq 3(R_2(X) - R_1(X))$$

将一次伸展中的所有旋转阶段求和，课件得到：

$$\text{伸展到节点 } X \text{ 的摊还时间} \leq 3(R(T) - R(X)) + 1 = O(\log N)$$

这把单次可能很长的伸展，转化为一段操作序列上的对数级摊还界。

3. 架构/机制与执行流程

3.1 AVL 更新与平衡恢复

1. 按 BST 规则插入或删除。
2. 沿更新路径向根回溯，更新高度并计算 BF。
3. 若所有节点满足 $BF \in \{-1, 0, 1\}$ ，结束。
4. 找到失衡节点 A ，按路径判断 LL、RR、LR 或 RL。
5. 执行一次单旋转或两次双旋转，保持中序顺序。
6. 继续检查祖先节点的高度和 BF。

3.2 Splay 访问流程

1. 按 BST 规则查找目标节点 X 。
2. 判断 X 、父节点 P 、祖父节点 G 的关系。
3. 按 Zig、Zig-zag、Zig-zig 之一旋转。
4. 重复，直到 X 成为根。
5. 后续对邻近节点的访问通常更快。

3.3 删除流程

```
Find(X)
|
Splay(X) -> X 成为根
|
Remove(X) -> 得到 TL、TR
|
FindMax(TL) -> 最大元素成为 TL 根且无右孩子
|
将 TR 接为该根的右孩子
```

4. 高频考点与常见坑点

- **空树高度约定**：课件明确取 $h(\varnothing) = -1$ 。若改用空树高度 0，递推初值和节点高度表达会整体平移，不能混用。
- **BF 符号方向**：本课件使用 $BF = h_L - h_R$ 。有些教材使用相反符号，判断 LL/RR 时必须先确认约定。
- **旋转类型按路径判断**：RR 是右-右，LL 是左-左；LR、RL 需要双旋转。不要只看新节点是左孩子还是右孩子。
- **AVL 的核心保证是高度界**：普通 BST 的搜索可能为 $O(N)$ ，AVL 通过高度平衡将高度控制为 $O(\ln n)$ 。
- **AVL 删除后不能只检查被删除节点**：祖先节点的高度和 BF 也可能改变；多个 BF 可能变化，即使不需要重构树。
- **Splay 不保证单次操作对数时间**：一次访问仍可能需要 $O(N)$ ；课件保证的是连续操作序列的摊还界 $O(M \log N)$ 。
- **Splay 不能只做逐层单旋转**：课件给出这种做法的坏例，可能导致 $O(N^2)$ ；必须使用 Zig、Zig-zag、Zig-zig。
- **摊还界不等于平均情况界**：摊还分析不涉及概率，不能把 $O(\log N)$ 直接解释为随机输入下的期望时间。
- **聚合分析的关键计数**：MultiPop 一次可能弹出很多元素，但每个元素最多被 Push 一次、弹出一次，总成本仍为线性。
- **记账法必须保证 credit 不透支**：对任意前缀都要满足 $\sum \hat{c}_i \geq \sum c_i$ ，否则摊还成本没有支付依据。
- **势能函数需要初始势能最小且通常非负**：否则边界项 $\Phi(D_n) - \Phi(D_0)$ 可能破坏总成本上界。
- **Splay 势能使用子树大小**：课件使用 $\Phi(T) = \sum \log S(i)$ ，不能直接把树高代入相同推导。
- **Splay 删除顺序不能颠倒**：必须先把 X 伸展到根，再处理左右子树；左子树取最大元素后接右子树，才能保持 BST 顺序。
- **区分复杂度变量**： n 、 N 可表示节点规模， M 表示连续操作数， h 表示树高； $O(M \log N)$ 是序列总时间界，不是每次操作的最坏时间界。